

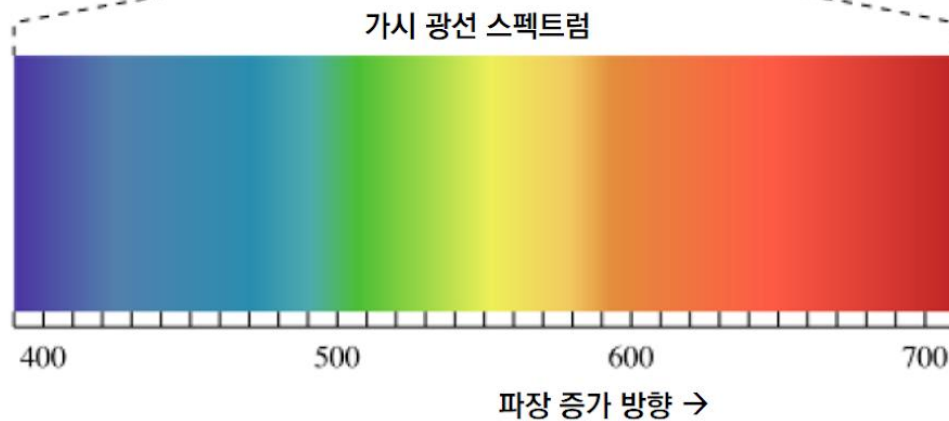
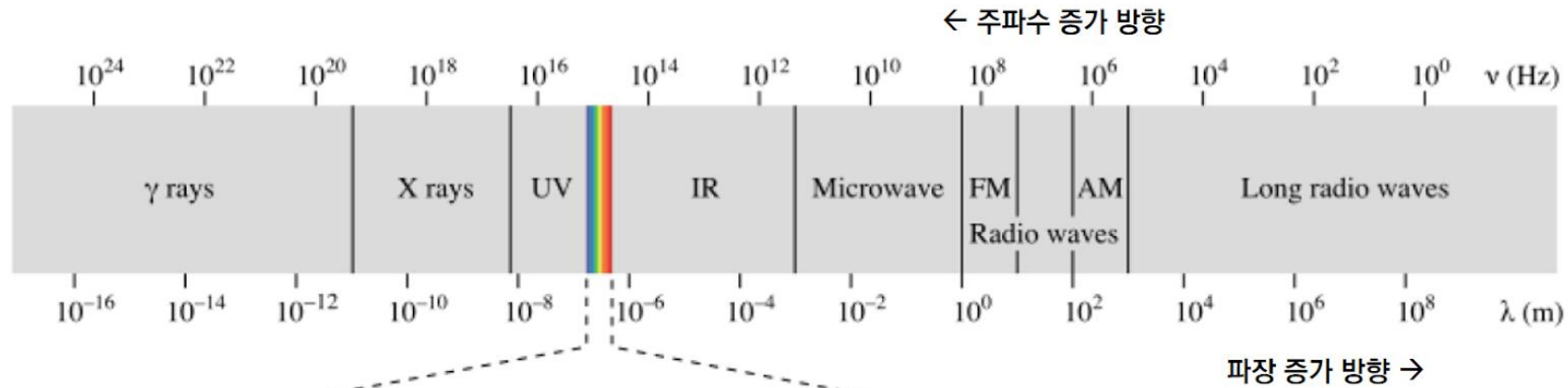
색과 조명

동명대학교 게임공학과

강영민

가시광선

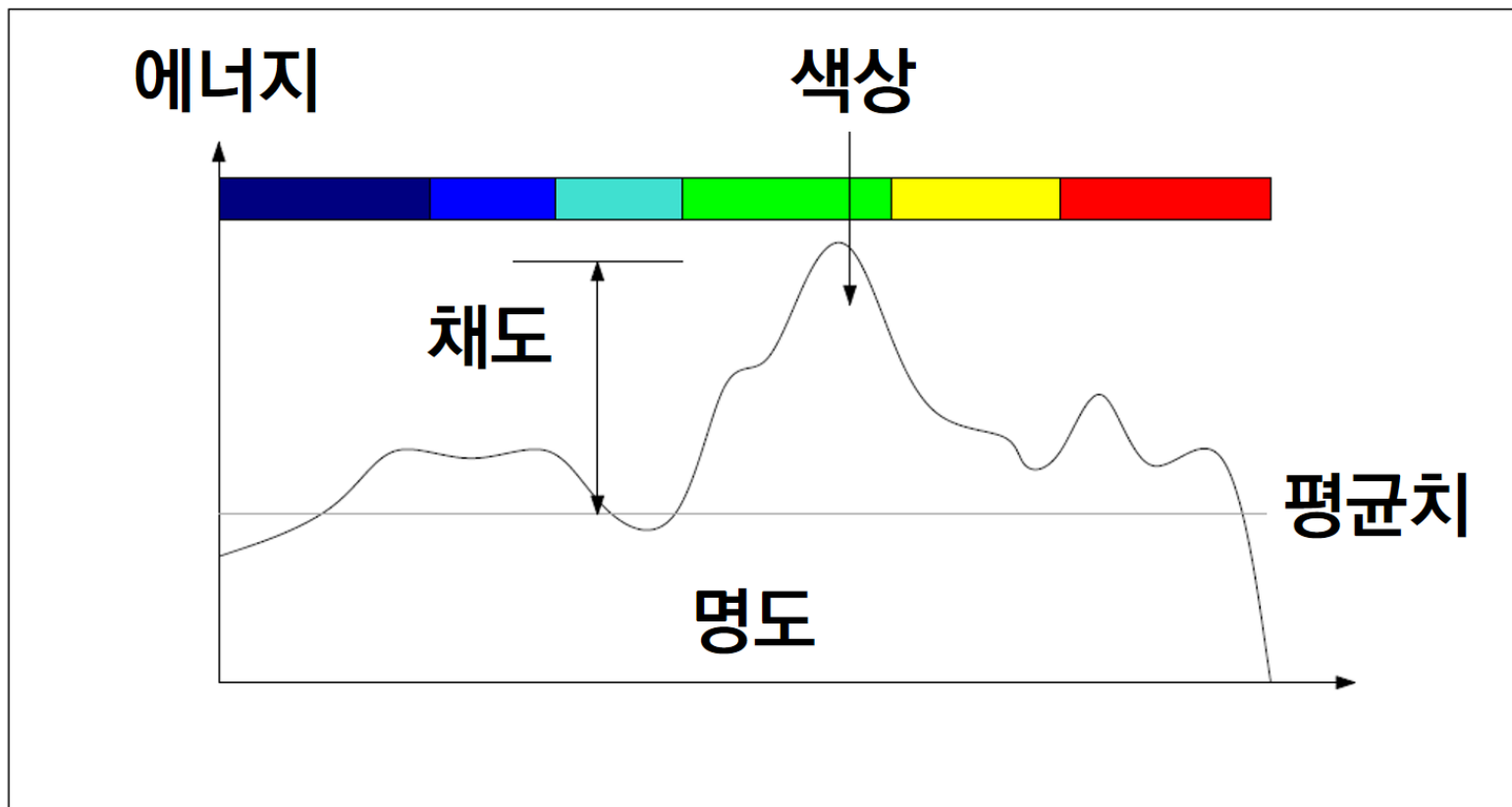
- 전자기파와 가시광선



390 nm에서 720 nm의 범위

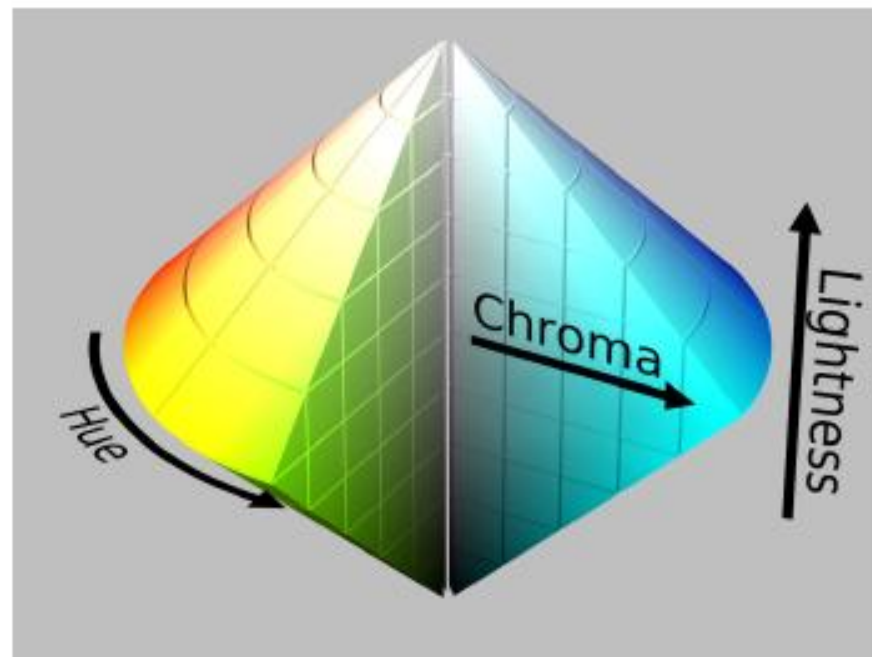
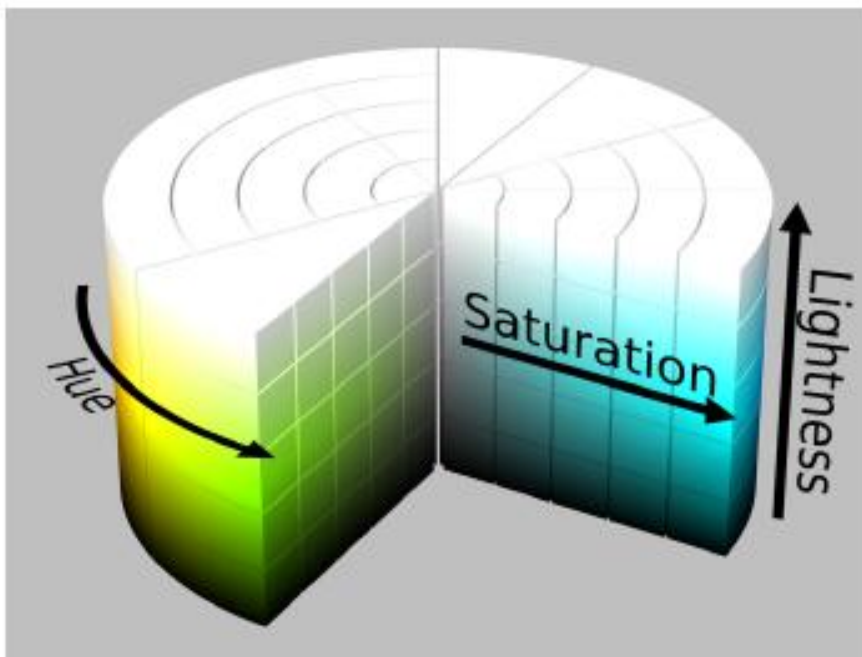
색에 대한 이해

- 스펙트럼 별 에너지와 색의 감각



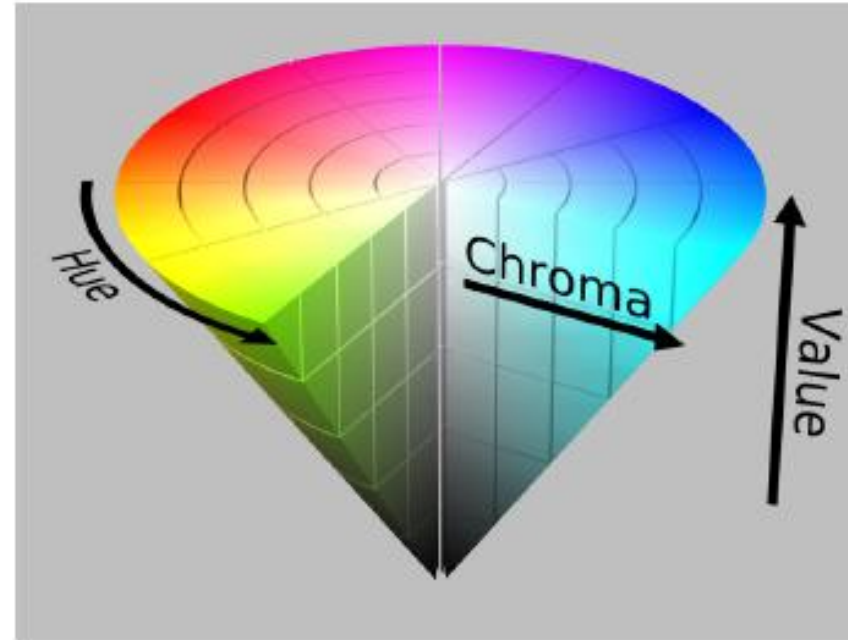
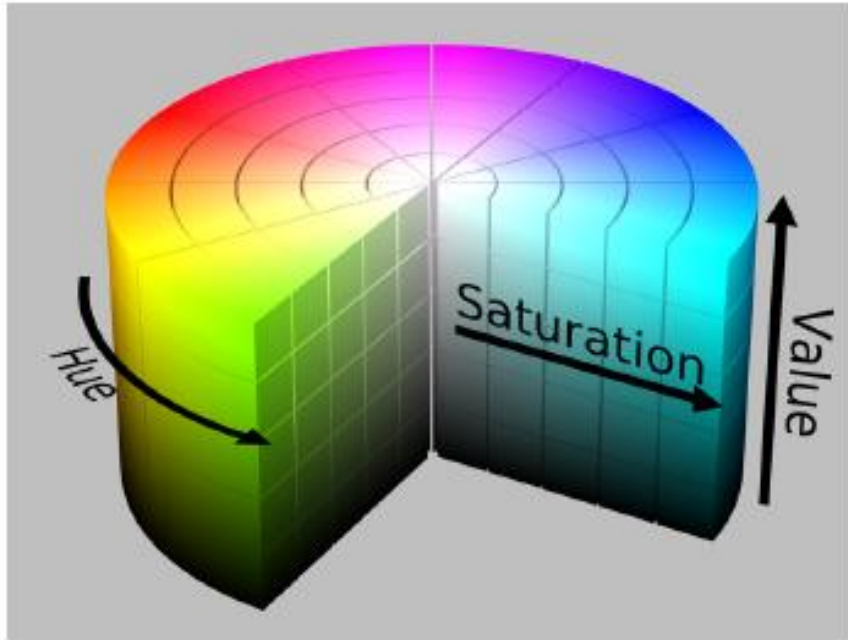
색모델

- HSL – Hue, Saturation, Lightness



색모델

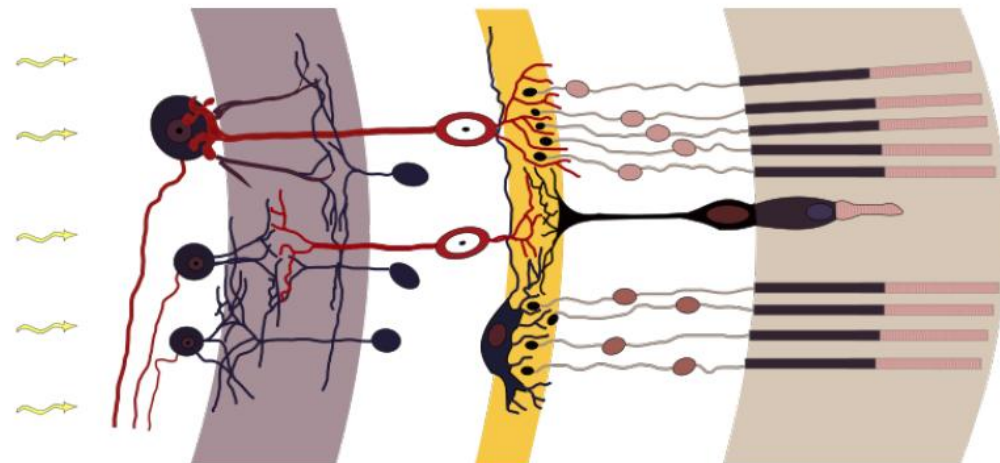
- HSV – Hue, Saturation, Value



HSL의 'L'과 HSV의 'V' 모두 가장 낮은 값일 때는 검정색을 표현한다는 점에서는 같다. 하지만 'L'이 가장 큰 값에 이르면 색이 흰색이 되는데 반해, 'V'가 최대치에 이르면 가장 채도가 높은 선명한 색이 된다.

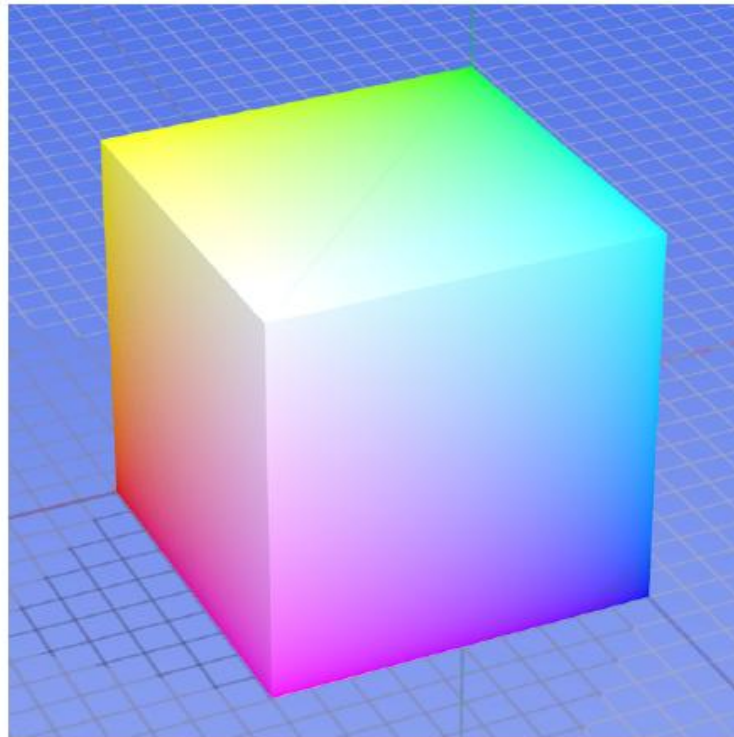
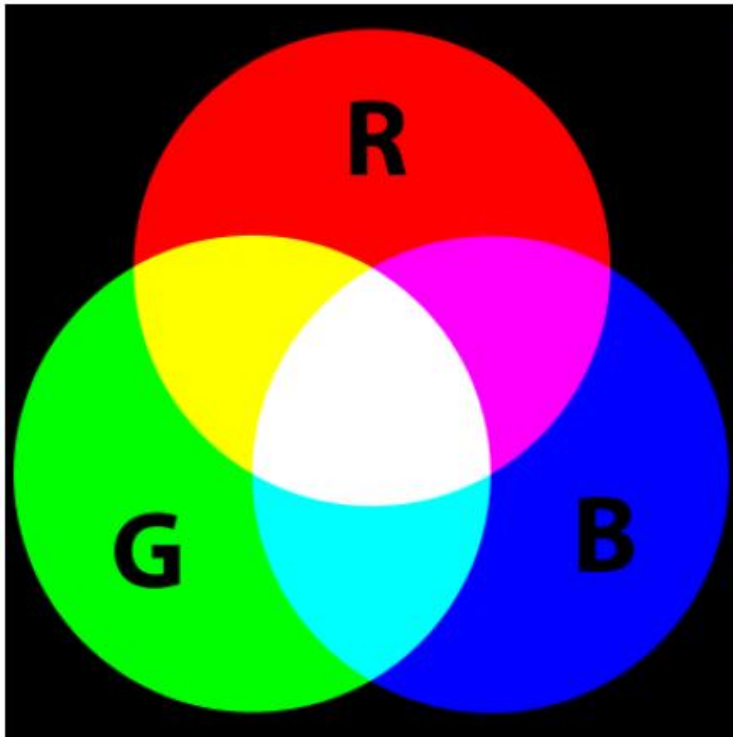
RGB 색모델과 삼중 자극 이론

- RGB 색 모델
 - 삼중자극(三重刺激, tristimulus) 이론에 근거하
 - 하나의 색을 세 종류의 색의 합성으로 보는 것
 - 요소가 되는 세 색은 빨강, 녹색, 파랑: R, G, B
 - 색을 인식하는 원추 세포가 이 세 가지 색상에 가장 민감하게 반응
- 눈
 - 각막(角膜, cornea)을 통해 들어온 빛
 - 렌즈의 역할을 하는 수정체^{lens}
 - 망막^{retina}에 상을 맺게 함
 - 망막의 세포
 - 명암을 인식하는 막대^{rod} 세포
 - 색상을 인지하는 원추^{cone} 세포
 - 원추 세포가 빨강, 녹색, 파랑에 민감하게 반응



RGB 색모델 – 가산혼합 모델

- RGB 모델은 그림과 같이 요소 색을 합쳤을 때 명도가 높아지는 가산혼합 모델
- 요소색: Red, Green, Blue → RGB (에너지가 합쳐지므로 밝은 색을 인지하게 됨)

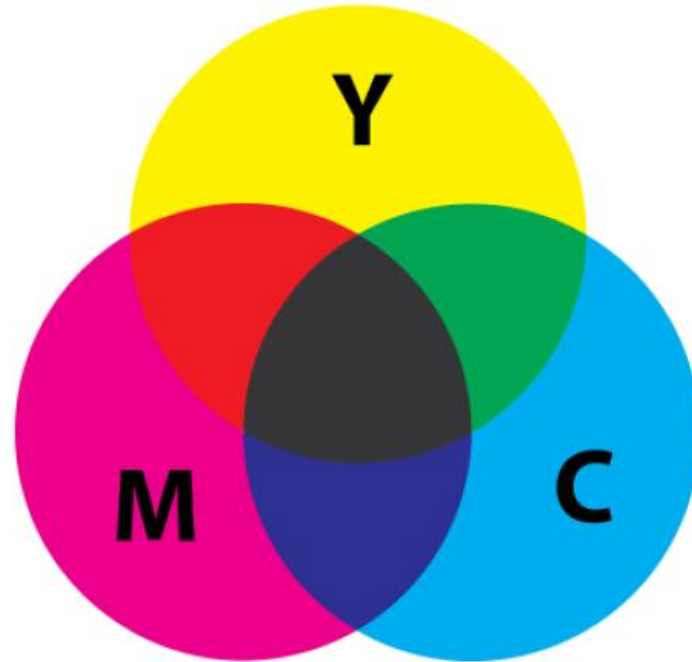


감산혼합 모델 - CMYK

CMYK 모델은 그림과 같이 요소 색을 합쳤을 때 명도가 낮아지는 감산혼합 모델

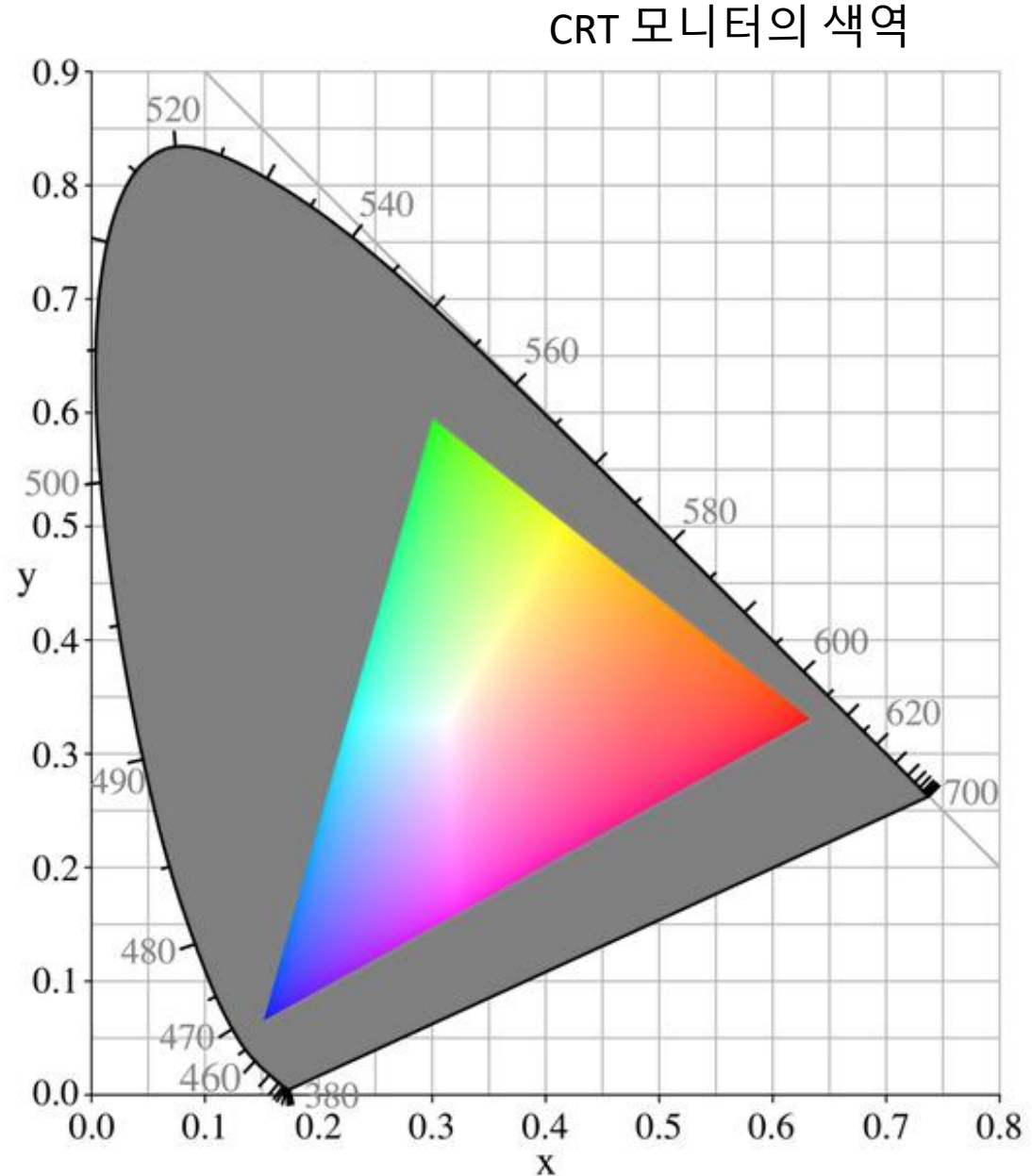
- 요소색: Cyan, Magenta, Yellow, Black → CMYK

- 잉크 모델
- 인쇄에서 사용됨

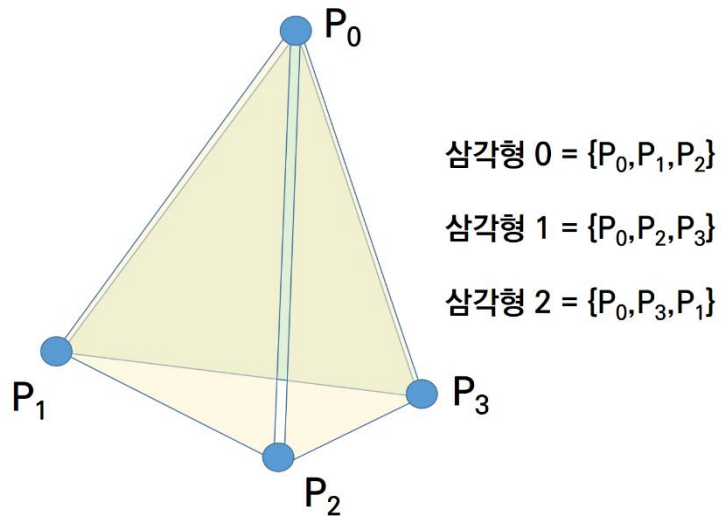


색역 – color gamut

색 모델에 따라서 표현할 수 있는 색의 범위가 제한된다. 이를 색의 범위, 혹은 색역^{color gamut}이라고 부른다. 색역의 표현은 국제조명위원회^{CIE}에서 정한 CIE 1931 색공간으로 표현하는 것이 일반적이다. 오른쪽 그림은 색공간 내에서 CRT 모니터가 갖는 색역을 보이고 있다.

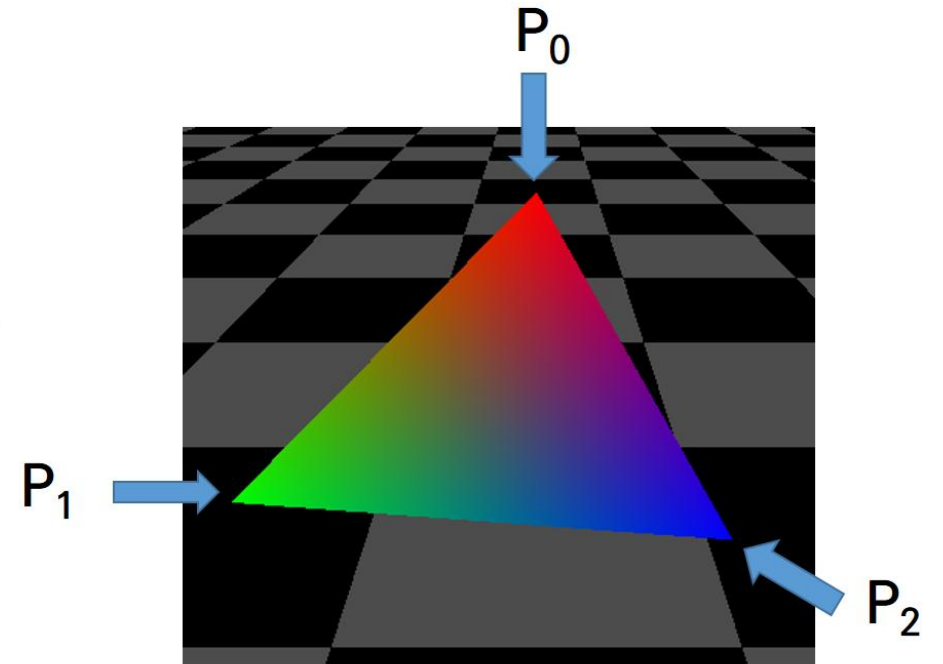


단순한 색상 지정

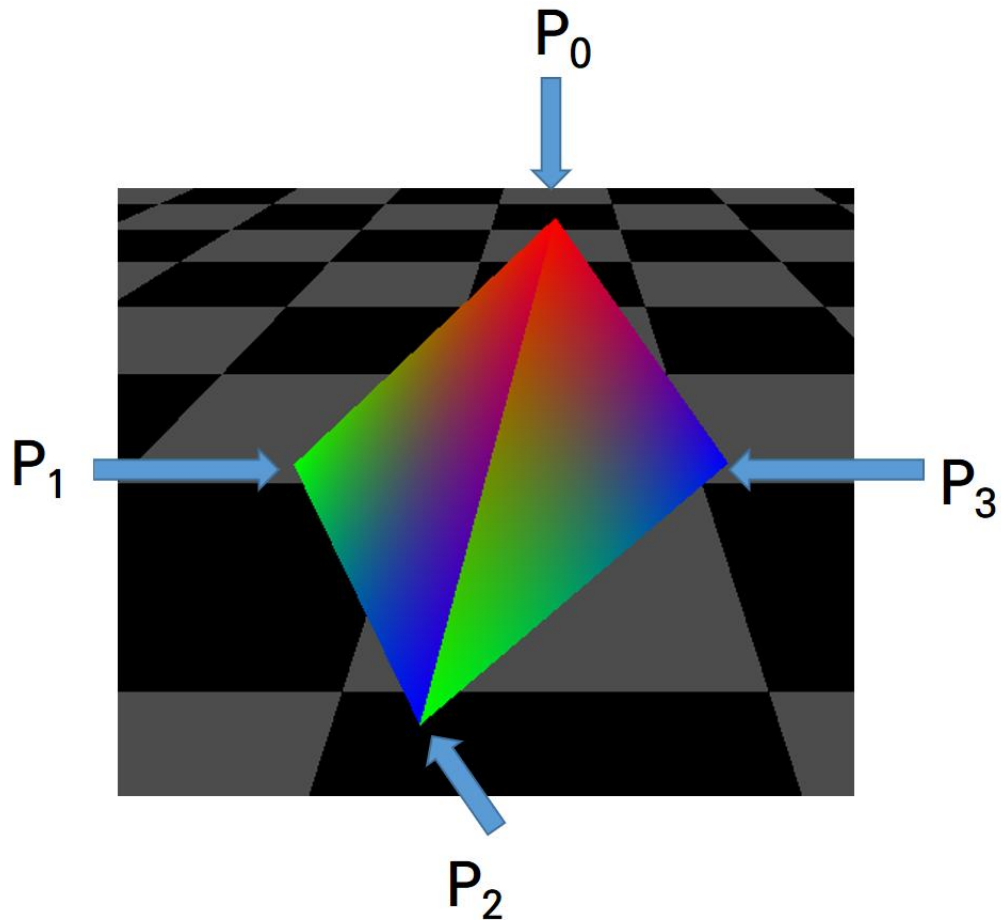


P0 = [0, 5, 0]
P1 = [-3, 1, 3]
P2 = [3, 1, 3]
P3 = [0, 1, -3]

```
glBegin(GL_TRIANGLES)  
glColor3f(1, 0, 0)  
glVertex3fv(P0)  
glColor3f(0, 1, 0)  
glVertex3fv(P1)  
glColor3f(0, 0, 1)  
glVertex3fv(P2)  
glEnd()
```



단순한 색상 지정



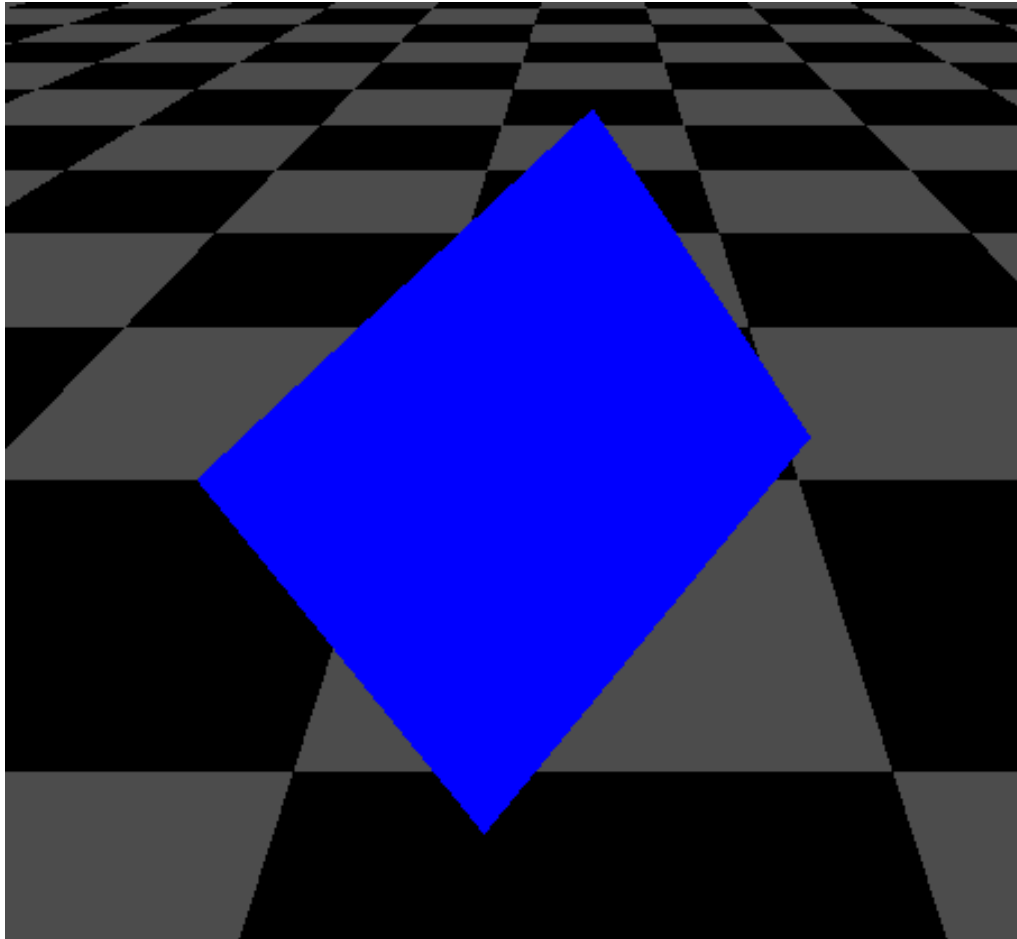
```
P0 = [0,5,0]
P1 = [-3,1,3]
P2 = [ 3,1,3]
P3 = [ 0,1,-3]
```

```
glRotatef(self.angle, 0, 1, 0)
self.angle += 1
glBegin(GL_TRIANGLES)
glColor3f(1, 0, 0)
glVertex3fv(P0)
glColor3f(0, 1, 0)
glVertex3fv(P1)
glColor3f(0, 0, 1)
glVertex3fv(P2)

glColor3f(1, 0, 0)
glVertex3fv(P0)
glColor3f(0, 1, 0)
glVertex3fv(P2)
glColor3f(0, 0, 1)
glVertex3fv(P3)

...
glEnd()
```

단순한 색상 지정

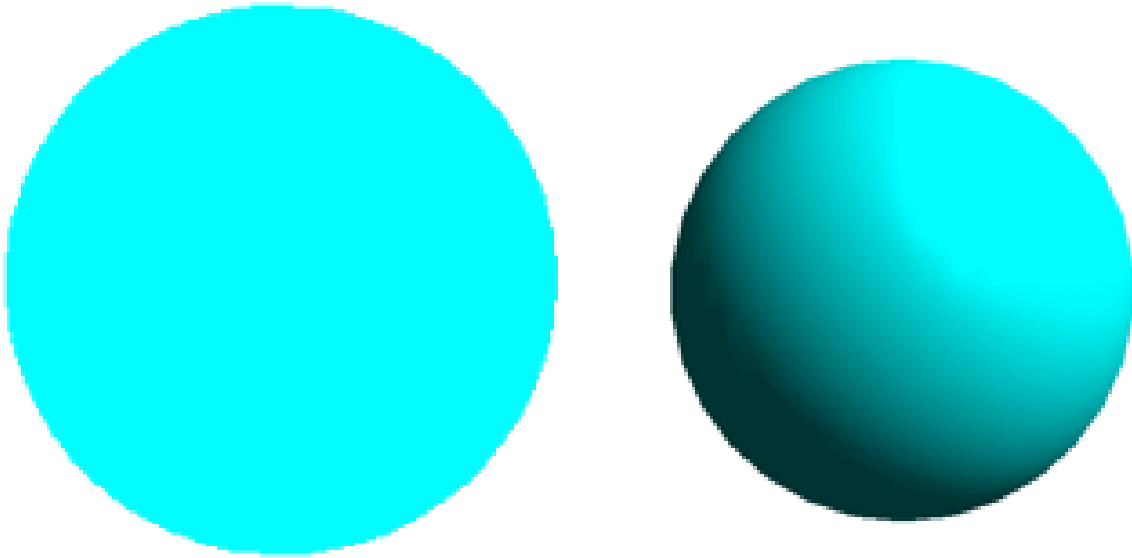


```
glShadeModel(GL_FLAT)
```

```
glBegin(GL_TRIANGLES)  
glColor3f(1, 0, 0)  
glVertex3fv(P0)  
glColor3f(0, 1, 0)  
glVertex3fv(P1)  
glColor3f(0, 0, 1)  
glVertex3fv(P2)
```

기본적으로
`glShadeModel(GL_SMOOTH)`
방식으로 동작

조명모델



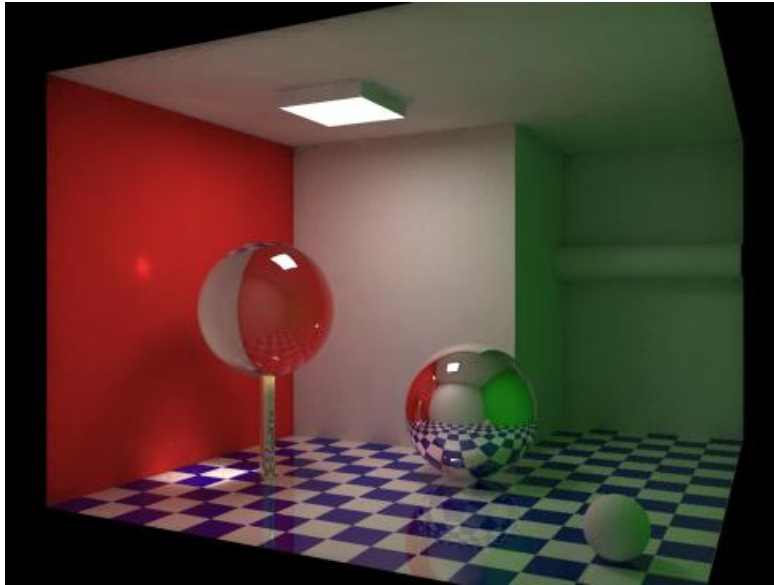
입체감이 없는 구와 입체감이 있는 구의 예

입체감은 무엇을 통해 얻을 수 있을까?

- 양안의 시차 (parallax)
- 그림자(shadow)
- 음영(shading)

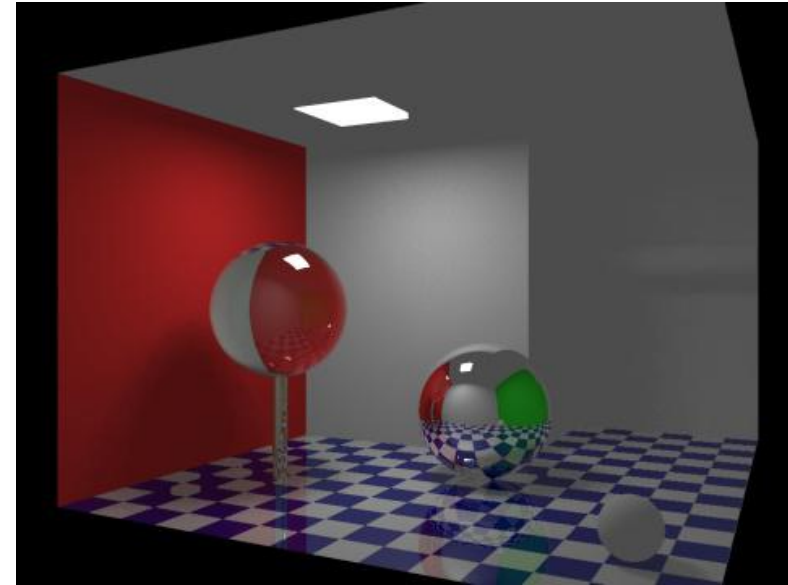
조명모델

전역조명(global illumination)



- 그림자(shadow)
- 음영(shading)

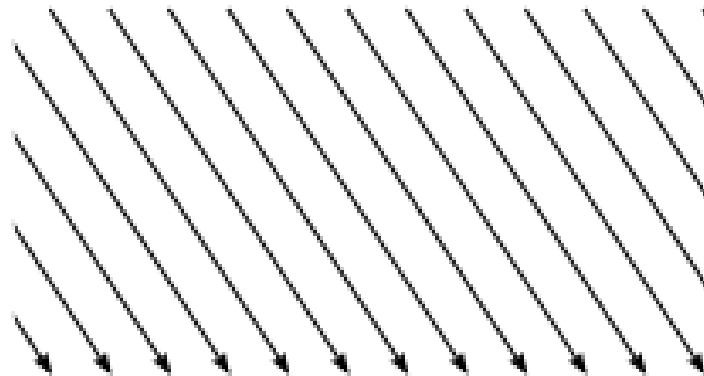
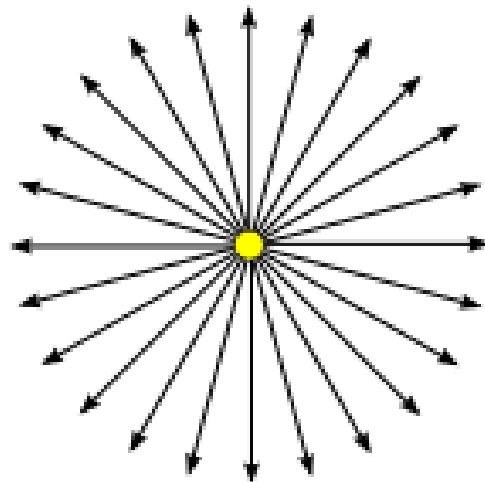
지역조명(local illumination)



- 음영(shading)
- * 그림자를 표현해도 제한적으로 표현

광원

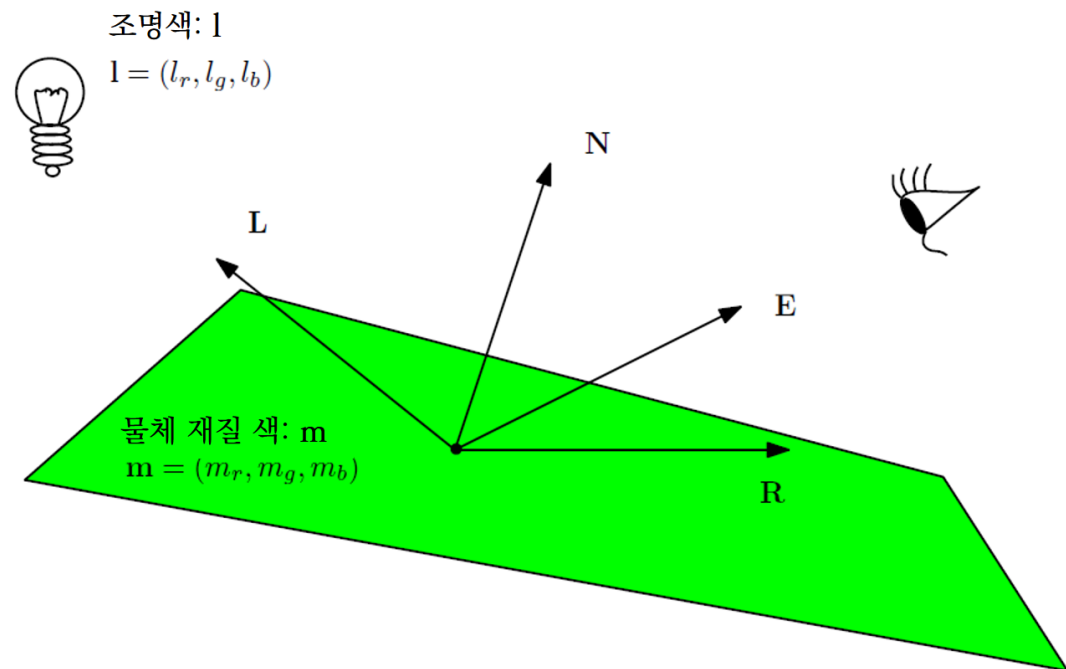
종류	특징
점 광원	광원의 위치와 색으로 결정. 전방향(omni-directional)으로 빛 진행
집중 광원	점광원과 동일하나 빛의 진행을 일부 입체각으로 빛을 제한
방향 광원	특정한 위치가 아니라 방향에 광원 존재
주변 광원	모든 곳에 동일하게 가해지는 빛



지역조명 – 뽕 모델

표면의 밝기 = 다음 밝기 요소들의 합

- 난반사 (diffuse reflection)
- 정반사 (specular reflection)
- 주변광 반사 (ambient reflection)



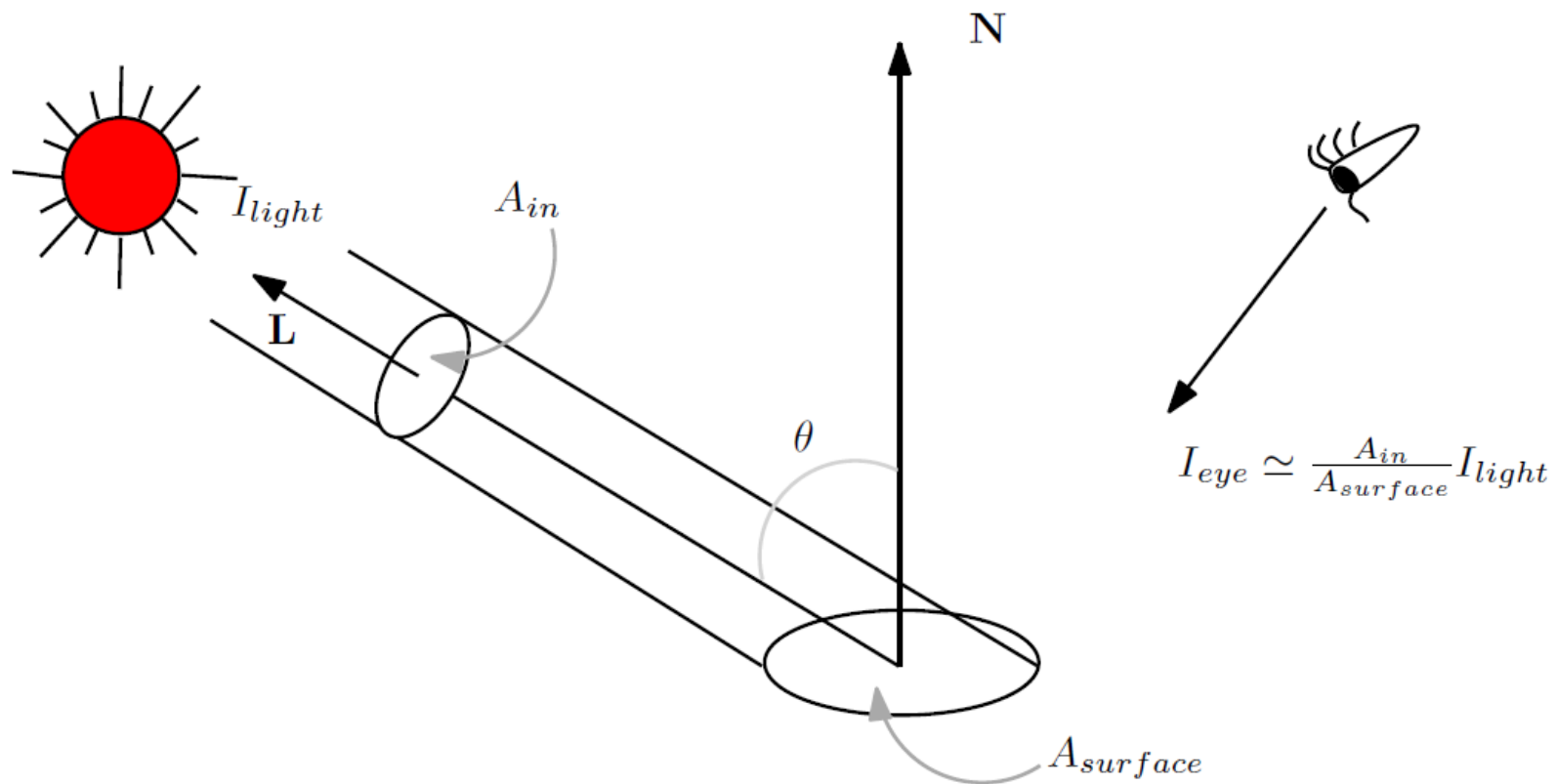
계산에 필요한 요소들

- 광원으로 향하는 벡터 $\mathbf{L}$
- 카메라(시점)을 향하는 벡터 $\mathbf{E}$
- 법선 벡터 $\mathbf{N}$
- 이상적인 반사 방향 $\mathbf{R}$

$$\kappa = \mathbf{l}_a \otimes \mathbf{m}_a + I_d \mathbf{l}_d \otimes \mathbf{m}_d + I_s \mathbf{l}_s \otimes \mathbf{m}_s$$

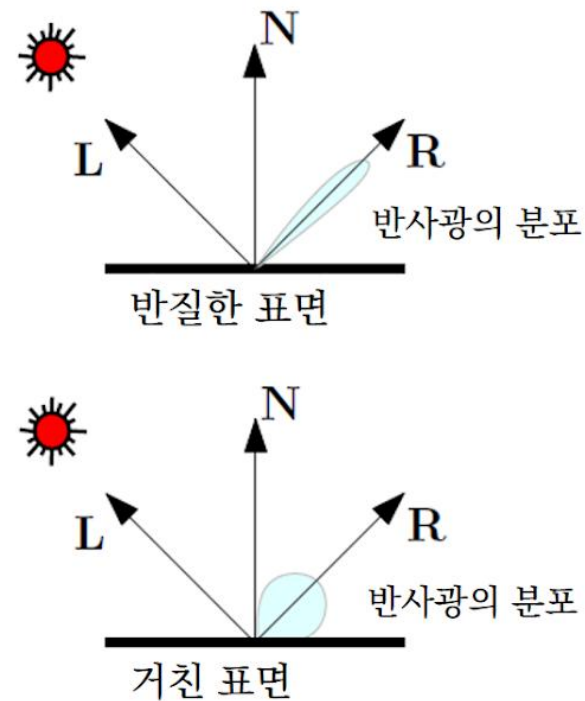
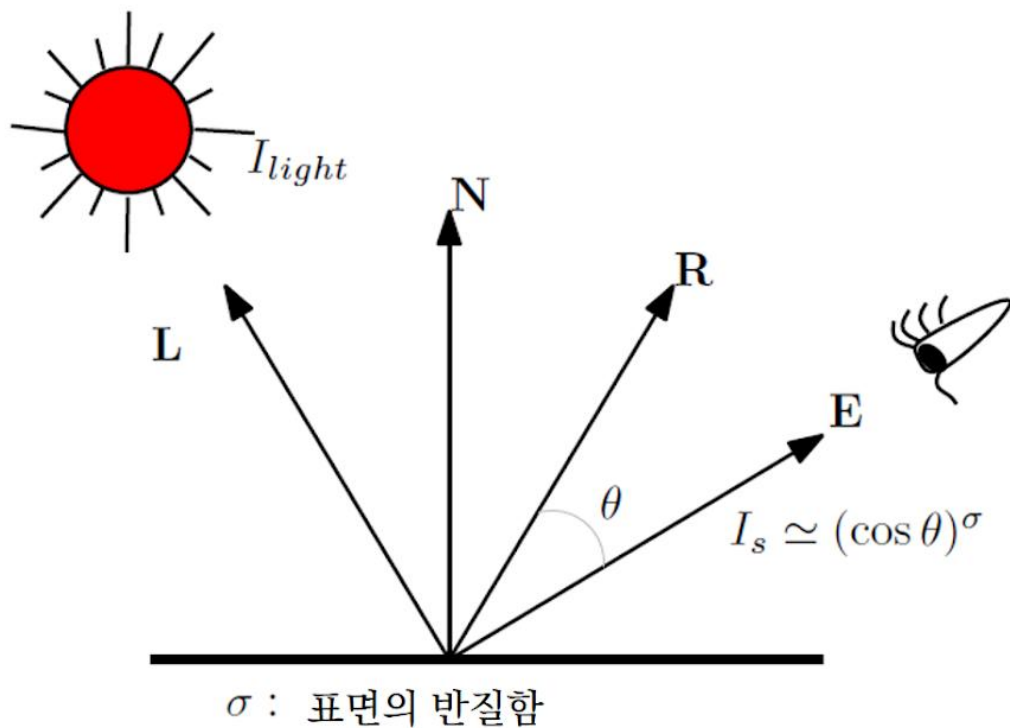
난반사의 계산

$$I_d = \cos \theta = \mathbf{L} \cdot \mathbf{N}$$



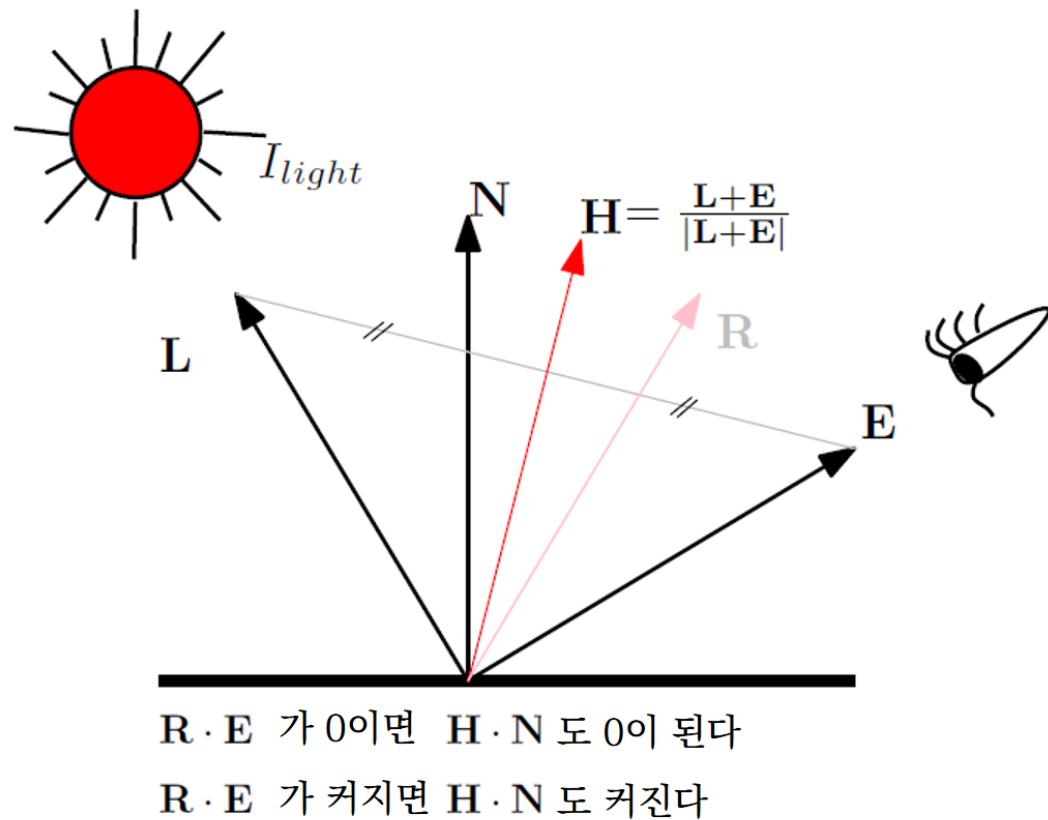
정반사의 계산

$$I_s = \cos\theta = (\mathbf{R} \cdot \mathbf{E})^\sigma$$



수정된 phong 모델 – 블린 모델

$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{E}}{|\mathbf{L} + \mathbf{E}|}$$



$$\kappa = \mathbf{l}_a \otimes \mathbf{m}_a + (\mathbf{L} \cdot \mathbf{N}) \mathbf{l}_d \otimes \mathbf{m}_d + (\mathbf{H} \cdot \mathbf{N})^\sigma \mathbf{l}_s \otimes \mathbf{m}_s$$

데이터 준비

```
mat_spec = [1.0, 1.0, 1.0, 1.0]
mat_diff = [0.0, 1.0, 1.0, 1.0]
mat_ambi = [1.0, 1.0, 1.0, 1.0]
mat_shin = [120]

lit_spec = [1.0, 1.0, 1.0, 1.0]
lit_diff = [1.0, 1.0, 1.0, 1.0]
lit_ambi = [0.0, 0.0, 0.0, 0.0]

light_pos = [1.0, 1.0, 1.0, 0.0]
```

광원과 재질 설정

```
def LightSet():
    glMaterialfv(GL_FRONT, GL_SPECULAR, mat_spec)
    glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_diff)
    glMaterialfv(GL_FRONT, GL_AMBIENT, mat_ambi)
    glMaterialfv(GL_FRONT, GL_SHININESS, mat_shin)

    glLightfv(GL_LIGHT0, GL_SPECULAR, lit_spec)
    glLightfv(GL_LIGHT0, GL_DIFFUSE, lit_diff)
    glLightfv(GL_LIGHT0, GL_AMBIENT, lit_ambi)

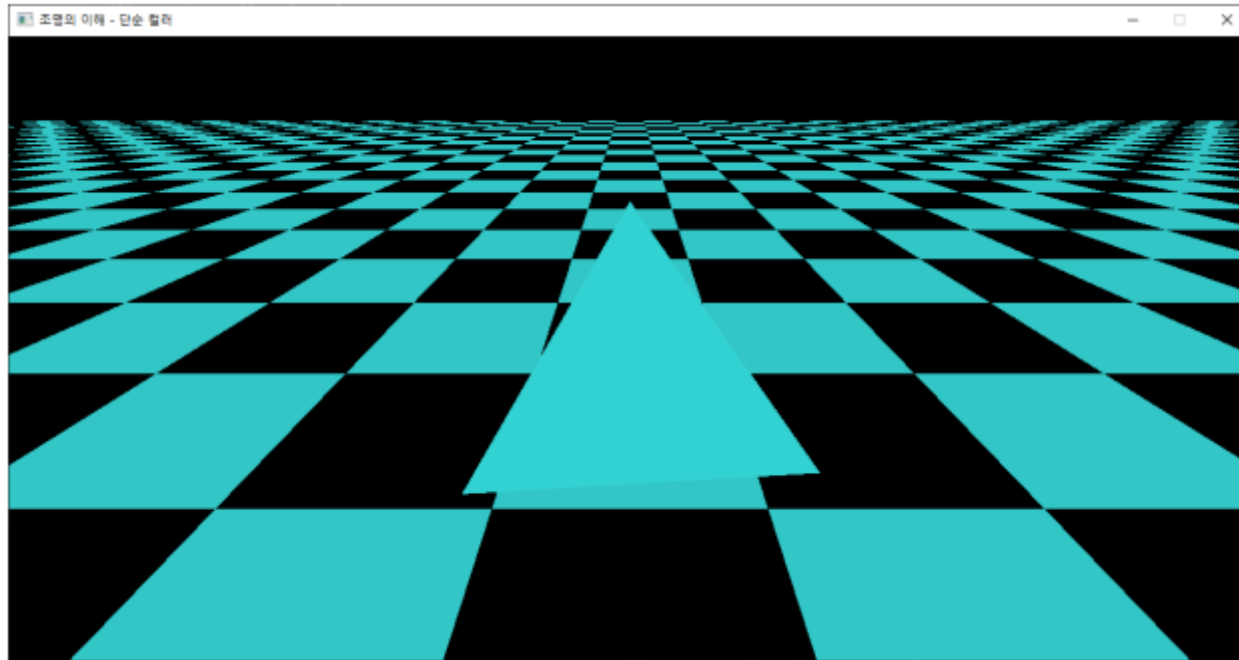
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)

def LightPositioning() :
    glLightfv(GL_LIGHT0, GL_POSITION, light_pos)
```

```
def initializeGL(self):
    ...
    LightSet()

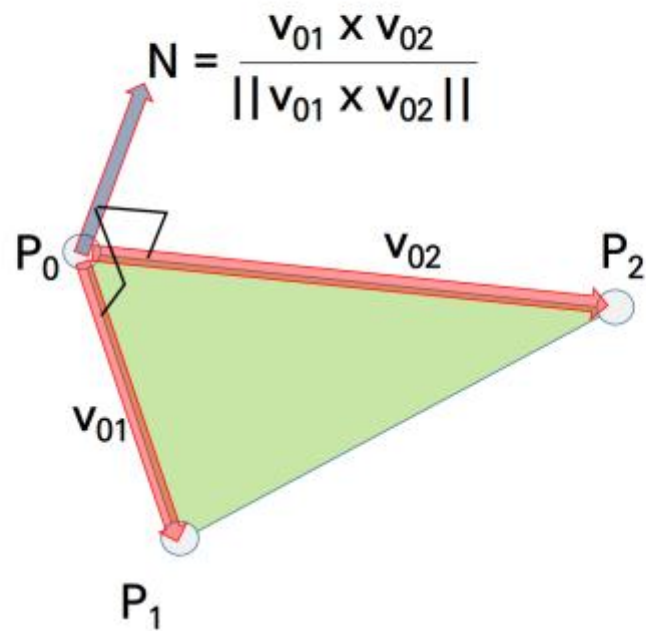
def paintGL(self):
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    gluLookAt(0,7,10, 0,2,0, 0,1,0)
    LightPositioning()
```

법선이 없을 때의 결과



법선

$$\mathbf{N} = \frac{\mathbf{v}_{01} \times \mathbf{v}_{02}}{\|\mathbf{v}_{01} \times \mathbf{v}_{02}\|}$$



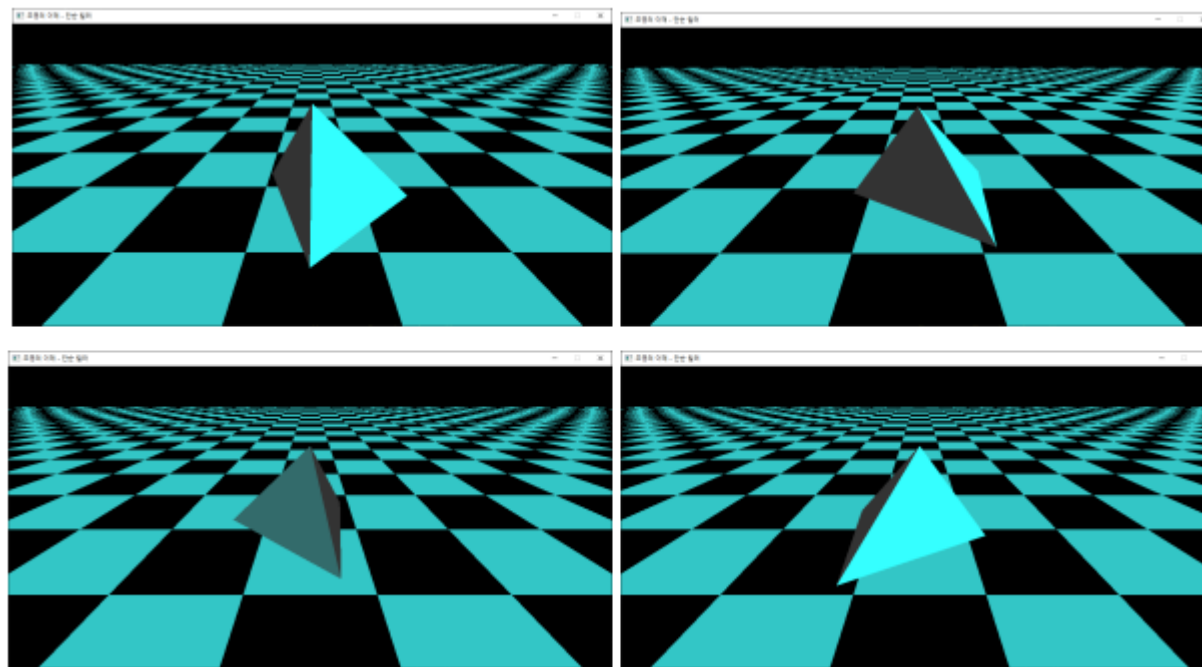
평면의 법선

```
def drawPlane():  
    n, w = 100, 500  
    # n: 체스판 한면의 정점수, w: 체스판 한면의 길이  
  
    d = w / (n-1) # 인접한 두 정점 사이의 간격  
  
    # 체스판 그리기  
    glNormal3f(0, 1, 0)  
    glBegin(GL_QUADS)  
    for i in range(n):  
        for j in range(n):  
            ...
```


임의의 삼각형의 법선

```
def paintGL(self):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)  
    glMatrixMode(GL_MODELVIEW)  
    ...  
    glBegin(GL_TRIANGLES)  
    u = P1-P0  
    v = P2-P0  
    N = np.cross(u, v)  
    norm = np.linalg.norm(N)  
    N = N / norm
```

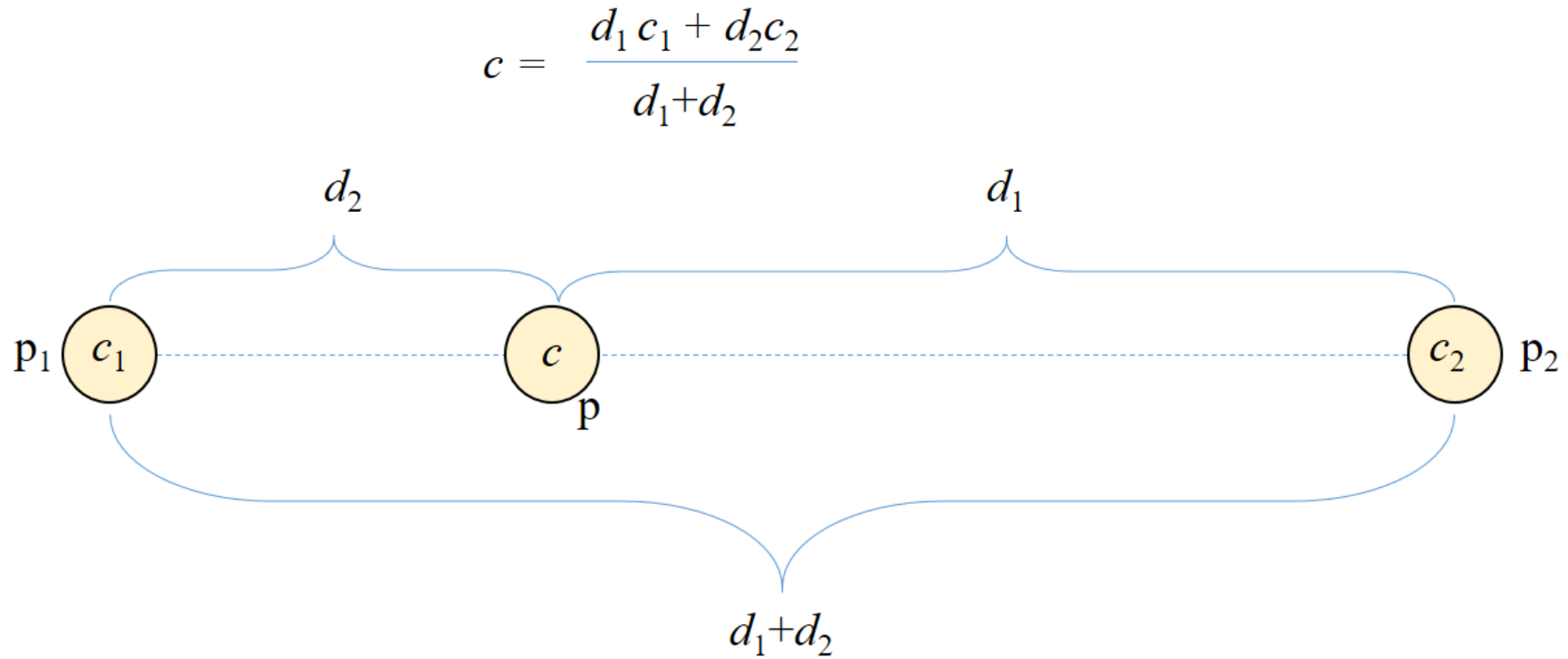
```
glNormal3fv(N)  
glVertex3fv(P0)  
glVertex3fv(P2)  
glVertex3fv(P3)
```



정점별 법선

보간(interpolation)

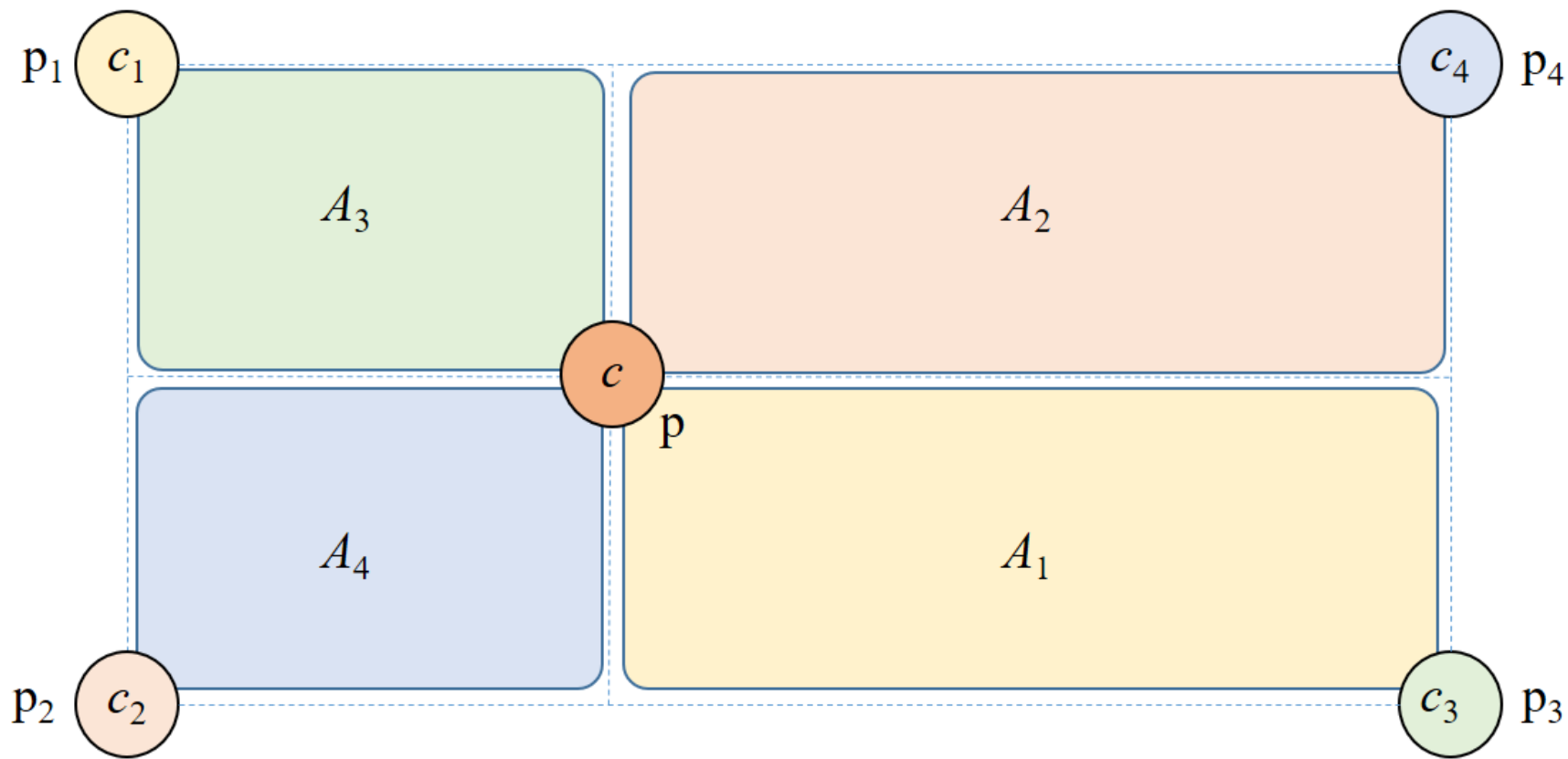
- 중간값 채우기



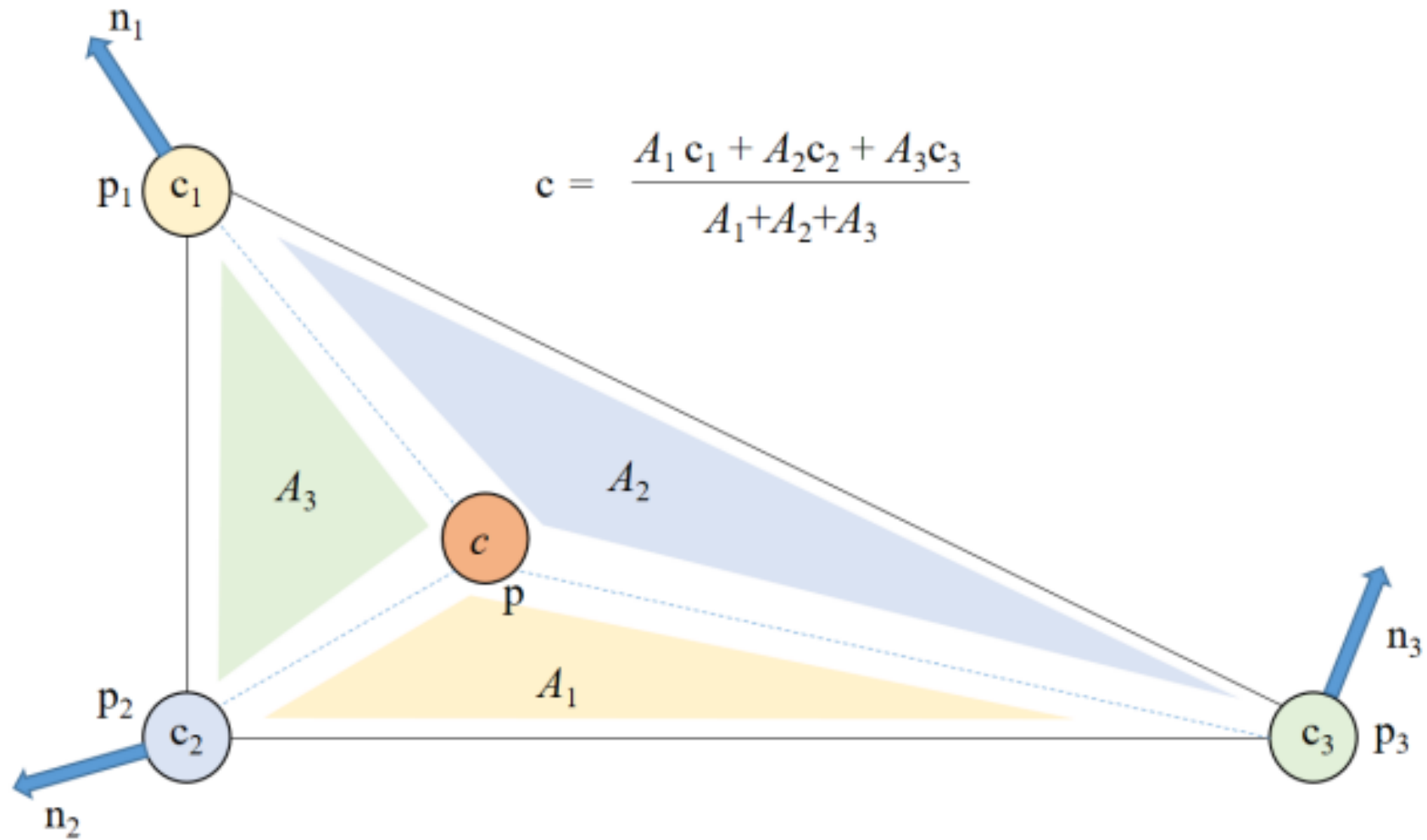
쌍선형 보간

- 2차원 보간

$$c = \frac{A_1c_1 + A_2c_2 + A_3c_3 + A_4c_4}{A_1 + A_2 + A_3 + A_4}$$

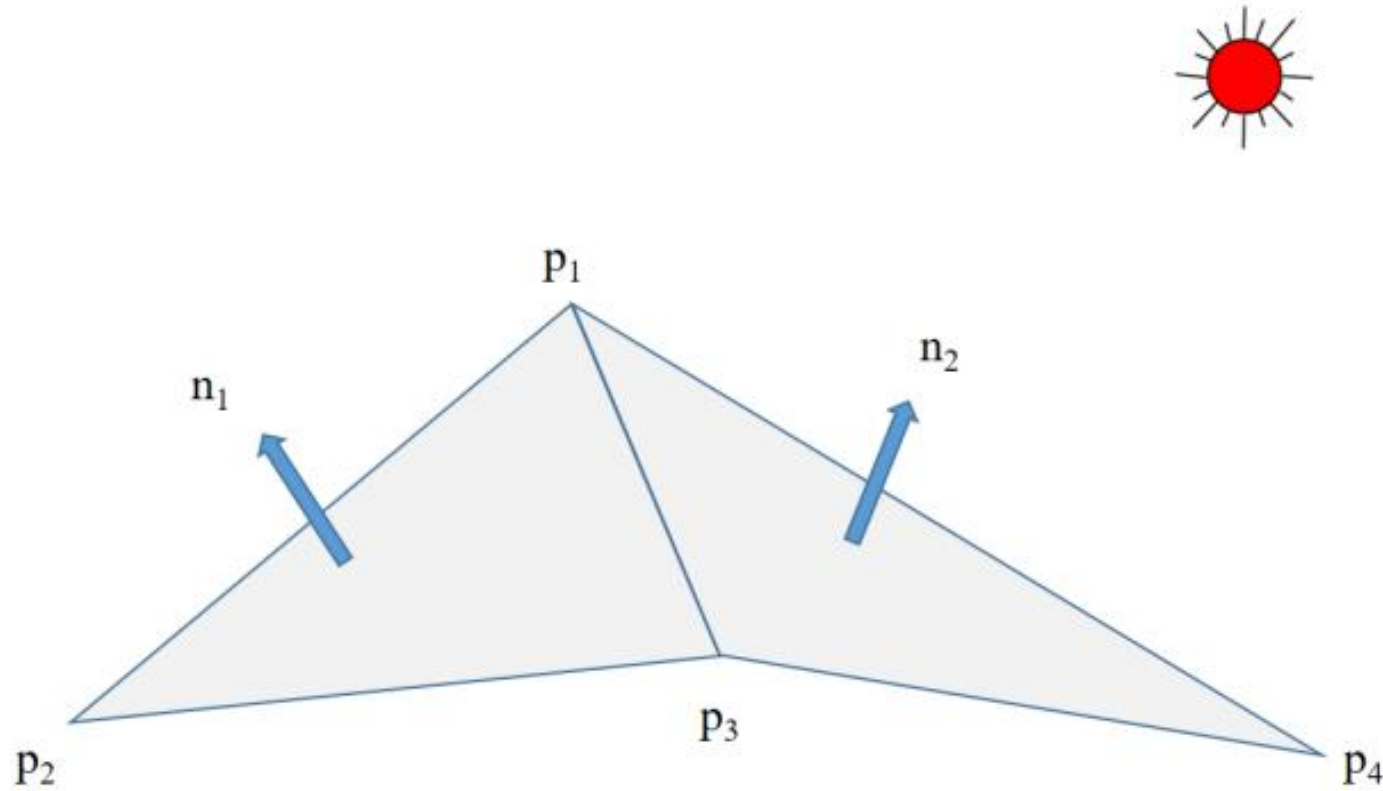


삼각형 내부의 보간

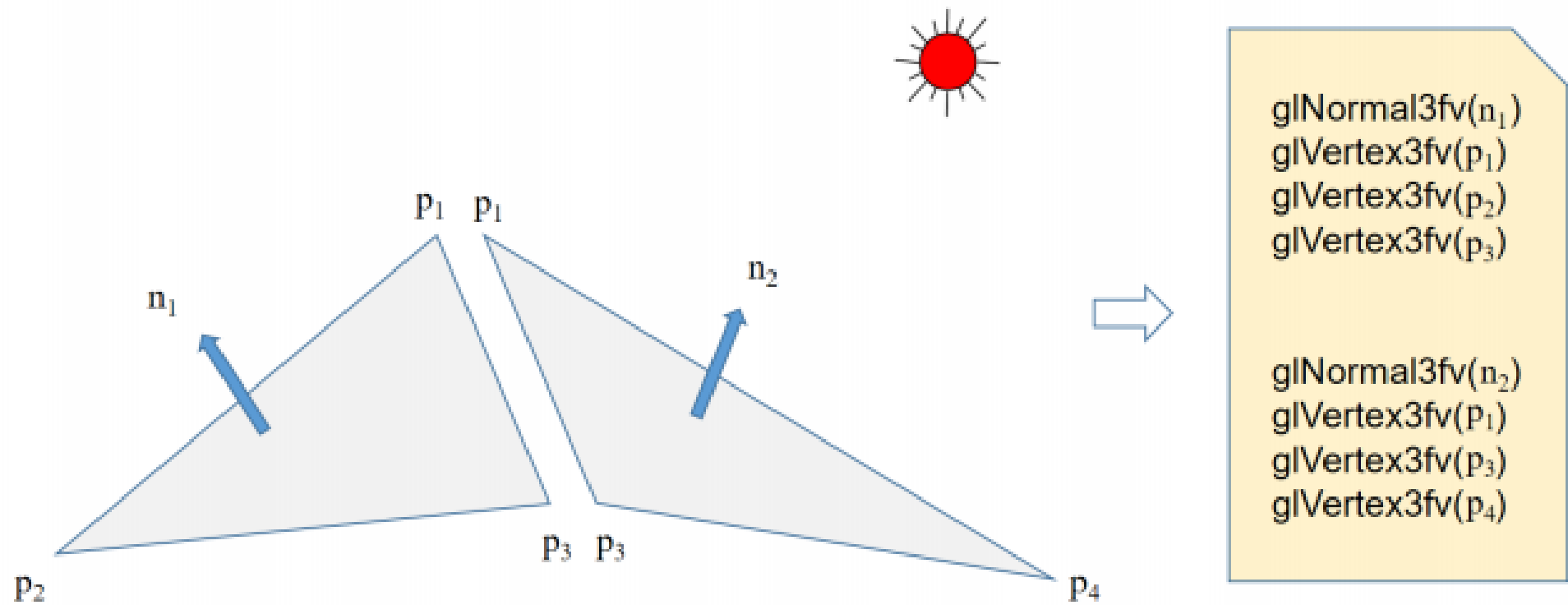


법선

- Phong 모델을 이용한 색상 계산에 반드시 필요 – p_1 의 법선은?



면별 법선 계산

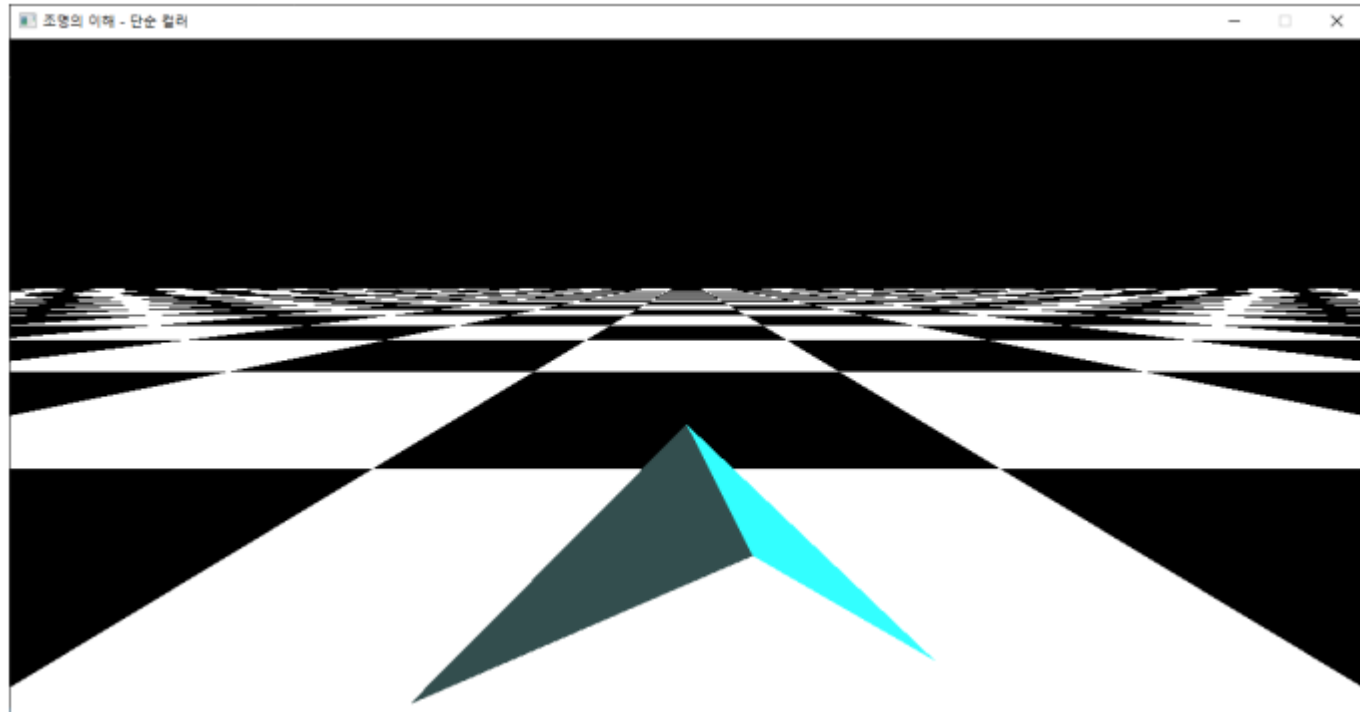


면별 법선 – 각진 모델

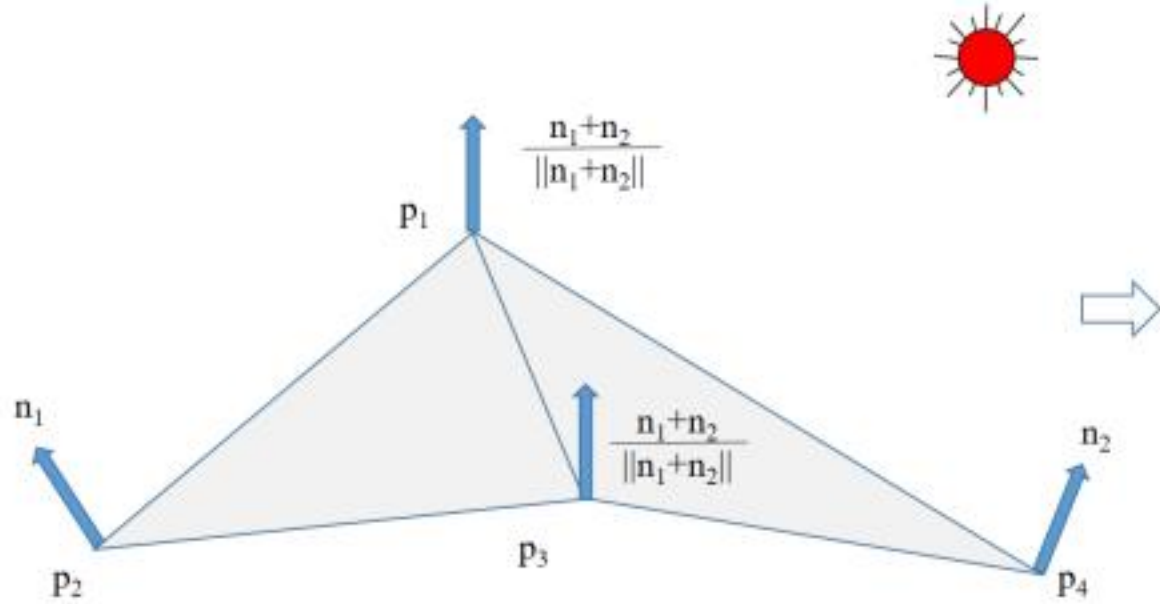
메시의 모양이 그대로 드러남 – Flat Shading

```
glEnable(GL_LIGHTING)
v = 1/math.sqrt(2)

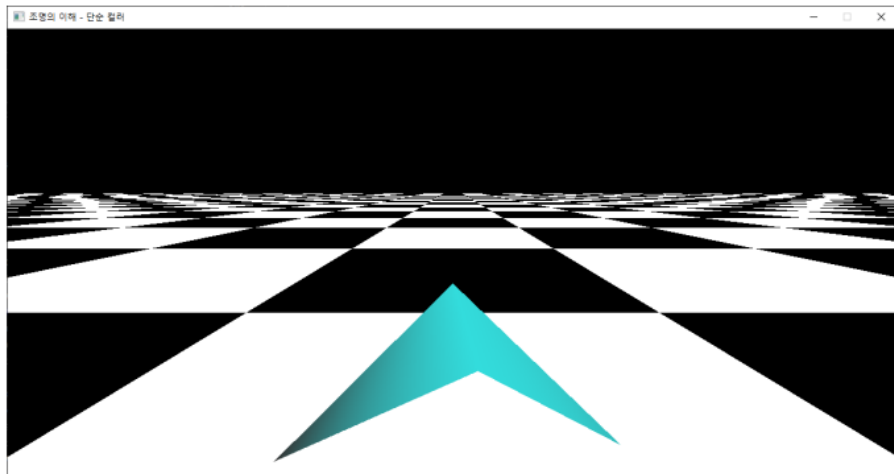
glBegin(GL_TRIANGLES)
glNormal3f (-v, v, 0)
glVertex3f(0,1,0)
glVertex3f(-1,0,0)
glVertex3f(0,1,1)
glNormal3f (v, v, 0)
glVertex3f(0,1,0)
glVertex3f(0,1,1)
glVertex3f(1,0,0)
glEnd()
```



한 점의 법선 – 참여한 모든 면의 법선을 모으기



```
glNormal3fv(  $\frac{n_1+n_2}{\|n_1+n_2\|}$  )  
glVertex3fv(p1)  
glNormal3fv(n1)  
glVertex3fv(p2)  
glNormal3fv(  $\frac{n_1+n_2}{\|n_1+n_2\|}$  )  
glVertex3fv(p3)  
  
glNormal3fv(  $\frac{n_1+n_2}{\|n_1+n_2\|}$  )  
glVertex3fv(p1)  
glNormal3fv(  $\frac{n_1+n_2}{\|n_1+n_2\|}$  )  
glVertex3fv(p3)  
glNormal3fv(n1)  
glVertex3fv(p4)
```



메시

면별 법선의 계산

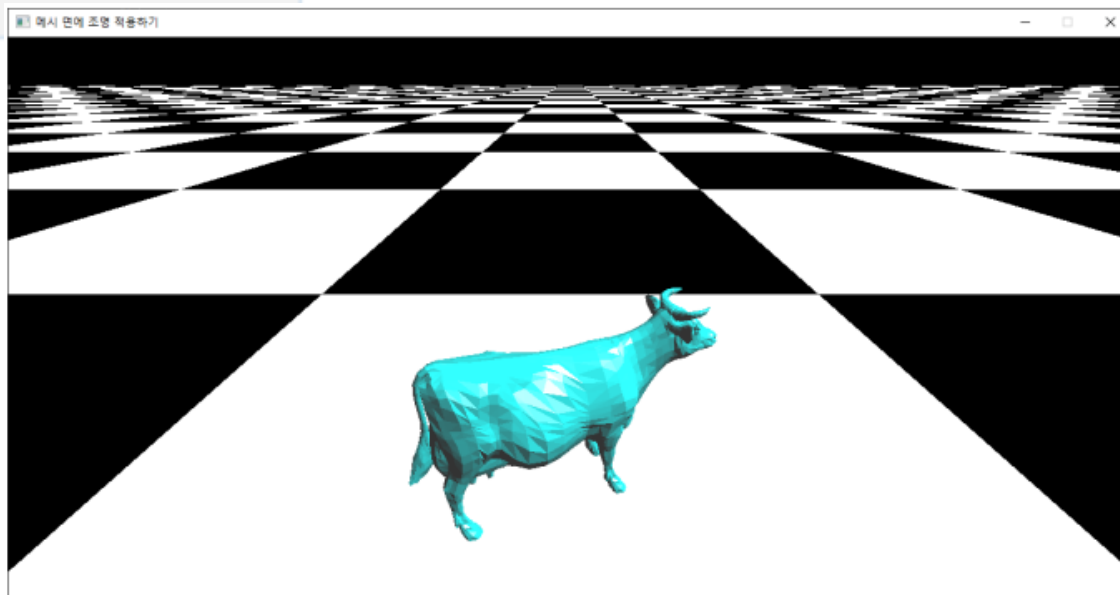
```
def loadData(self, filename):
    self.nF = int(next(inputfile))
    self.idxBuffer = np.zeros(shape=(self.nF*3, ), dtype=int)

    ### 법선벡터 저장을 위한 공간 준비
    self.normalBuffer = np.zeros(shape=(self.nF*3,), dtype=float )
```

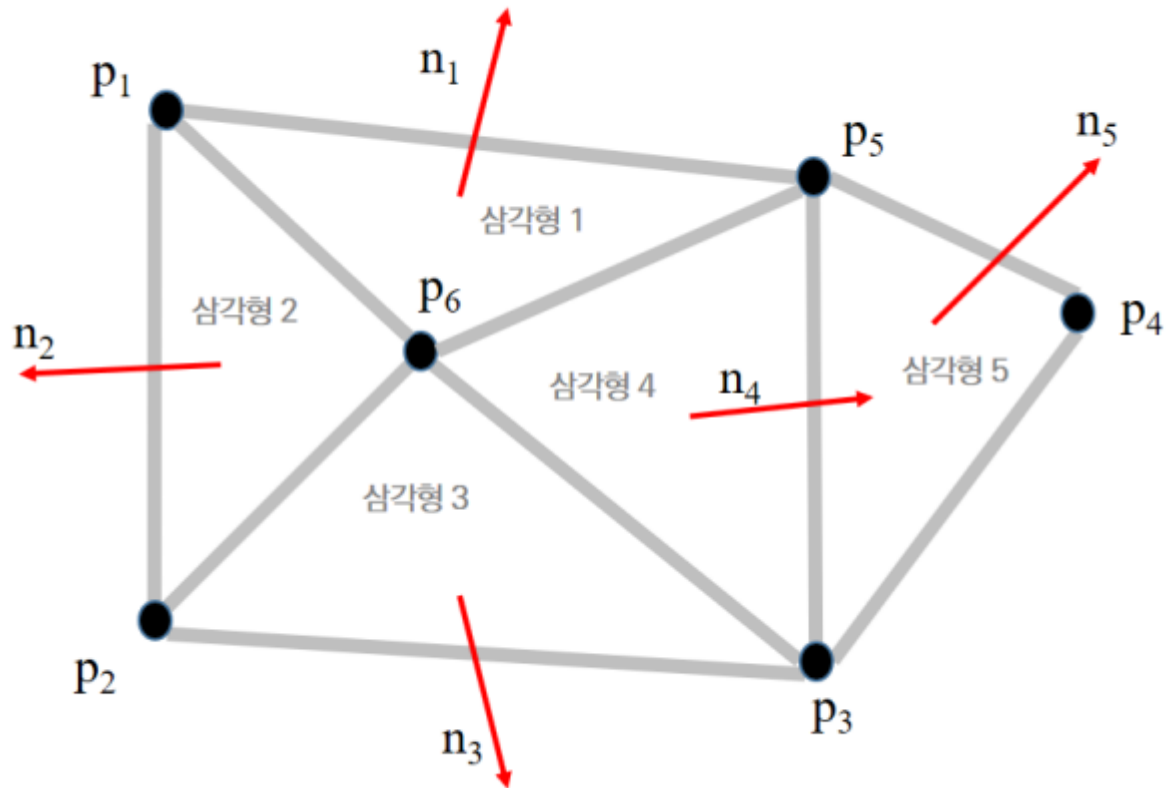
```
def loadData(self, filename):
    ...
    for i in range(self.nF):
        idx = next(inputfile).split()
        self.idxBuffer[i*3: i*3+3] = idx[1:4]
        index = self.idxBuffer[i*3: i*3+3]
        p0 = self.vertexBuffer[index[0]*3: index[0]*3 + 3]
        p1 = self.vertexBuffer[index[1]*3: index[1]*3 + 3]
        p2 = self.vertexBuffer[index[2]*3: index[2]*3 + 3]
        u = p1-p0
        v = p2-p0
        N = np.cross(u, v)
        norm = np.linalg.norm(N)
        N = N/norm
        self.normalBuffer[i*3: i*3+3] = N
```

면 그리기

```
def draw(self):  
  
    glBegin(GL_TRIANGLES)  
    for i in range(self.nF):  
        verts = self.idxBuffer[i*3: i*3+3]  
        glNormal3fv( self.normalBuffer[i*3: i*3+3] )  
        glVertex3fv( self.vertexBuffer[verts[0]*3 : verts[0]*3 +3] )  
        glVertex3fv( self.vertexBuffer[verts[1]*3 : verts[1]*3 +3] )  
        glVertex3fv( self.vertexBuffer[verts[2]*3 : verts[2]*3 +3] )  
    glEnd()
```



정점별 법선



정점별 법선을 저장할 공간

```
def loadData(self, filename):  
    with open(filename, 'rt') as inputfile:  
        self.nV = int(next(inputfile))  
  
        self.vertexBuffer = np.zeros(shape = (self.nV*3, ),  
                                       dtype=float)  
        ### 정점 별 법선벡터 저장을 위한 공간 준비  
        self.normalBuffer = np.zeros(shape = (self.nV*3, ),  
                                       dtype=float)
```

정점별 법선을 계산하기

```
def loadData(self, filename):
    with open(filename, 'rt') as inputfile:
        self.nV = int(next(inputfile))

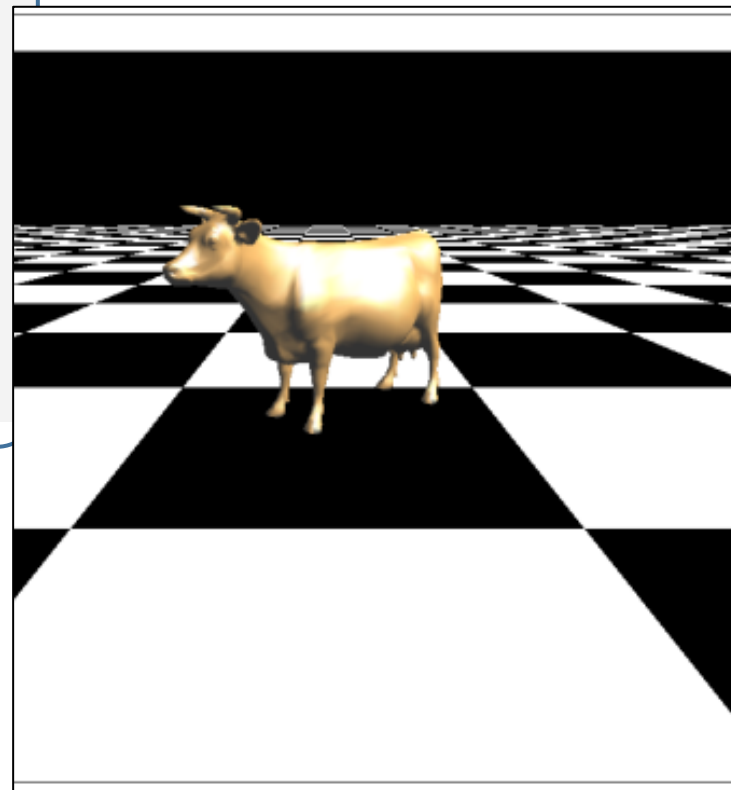
    ...

    for i in range(self.nF):
        idx = next(inputfile).split()
        self.idxBuffer[i*3: i*3+3] = idx[1:4]
        index = self.idxBuffer[i*3: i*3+3]
        p0 = self.vertexBuffer[index[0]*3: index[0]*3 + 3]
        p1 = self.vertexBuffer[index[1]*3: index[1]*3 + 3]
        p2 = self.vertexBuffer[index[2]*3: index[2]*3 + 3]
        u = p1-p0
        v = p2-p0
        N = np.cross(u, v)
        self.normalBuffer[index[0]*3: index[0]*3 + 3] += N
        self.normalBuffer[index[1]*3: index[1]*3 + 3] += N
        self.normalBuffer[index[2]*3: index[2]*3 + 3] += N

    for i in range(self.nV):
        N = self.normalBuffer[i*3: i*3 + 3]
        norm = np.linalg.norm(N)
        N = N/norm
        self.normalBuffer[i*3: i*3 + 3] = N
```

면 그리기 – 정점별 법선 설정

```
def draw(self):  
    glBegin(GL_TRIANGLES)  
    for i in range(self.nF):  
        verts = self.idxBuffer[i*3: i*3+3]  
        glNormal3fv( self.normalBuffer[verts[0]*3 : verts[0]*3 +3] )  
        glVertex3fv( self.vertexBuffer[verts[0]*3 : verts[0]*3 +3] )  
        glNormal3fv( self.normalBuffer[verts[1]*3 : verts[1]*3 +3] )  
        glVertex3fv( self.vertexBuffer[verts[1]*3 : verts[1]*3 +3] )  
        glNormal3fv( self.normalBuffer[verts[2]*3 : verts[2]*3 +3] )  
        glVertex3fv( self.vertexBuffer[verts[2]*3 : verts[2]*3 +3] )  
    glEnd()
```



광원

